

Paradigma Funcional – tipos, condicional e função

Fabio Lubacheski
fabio.lubacheski@mackenzie.br

Valores e tipos em Haskell

- **Valores** são as entidades básicas de uma linguagem de programação que são manipuladas durante a execução do programa.
- Um **tipo** é uma coleção de **valores** relacionados, por exemplo, em Haskell, o tipo **Bool** contém os dois valores lógicos **False** e **True**.

Bool valores lógicos : `True`, `False`

Char caracteres simples: `'A'`, `'B'`, `'?'`, `'\n'`

String sequências de caracteres: `"Abba"`, `"UB40"`

Int inteiros de precisão fixa (32 ou 64-bits): `142`, `-1233456`

Integer inteiros de precisão arbitrária: (apenas limitados pela memória do computador)

Float vírgula flutuante de precisão simples: `3.14154`, `-1.23e10`

Double vírgula flutuante de precisão dupla

Listas em Haskell

- Uma **lista** é uma sequência de **valores** do **mesmo tipo**, e é escrita colocando os elementos da sequência entre **colchetes** e separados por vírgula.

- Exemplos:

```
[False,True,False] :: [Bool]
```

```
['a','b','c','d'] :: [Char]
```

```
["bom","dia","Brasil"] :: [String]
```

```
[1,8,6,0,21] :: [Int]
```

```
[3.4,5.6,3.213] :: [Double]
```

- Em geral `[T]` é o tipo de listas cujos elementos são de tipo `T`
- A **lista vazia** `[]` admite qualquer tipo de lista `[T]`

Funções genéricas para listas - Módulo Prelude

λ head [1,2,3,4] obter o 1o elemento
1

λ tail [1,2,3,4] remover o 1o elemento
[2,3,4]

λ length [1,2,3,4,5] comprimento
5

λ take 3 [1,2,3,4,5] obter um prefixo
[1,2,3]

λ drop 3 [1,2,3,4,5] remover um prefixo
[4,5]

Funções genéricas para listas - Módulo Prelude

λ [1,2,3] ++ [4,5] concatenar
[1,2,3,4,5]

λ reverse [1,2,3,4,5] inverter
[5,4,3,2,1]

λ [1,2,3,4,5] !! 3 obter elemento no índice
4

λ sum [1,2,3,4,5] soma dos elementos
15

λ product [1,2,3,4,5] produto dos elementos
120

Funções genéricas para listas - Módulo Prelude

`λ maximum [3,2,6,4,1,2,3]` retorna o valor máximo
6

`λ minimum [3,2,6,4,1,2,3]` retorna o valor mínimo
1

`λ replicate 3 5` cria um lista do tamanho do 1º argumento
[5,5,5]

Tuplas em Haskell

- Uma **tupla** é uma sequência de **valores** de **diferentes tipos**, e é escrita colocando os elementos da sequência entre **parênteses** separados por vírgula.
- Exemplos:
`(False,True) :: (Bool,Bool)`
`(False,'a',True) :: (Bool,Char,Bool)`
`("Joel",16,5.34,True) :: (String,Int,Float,Bool)`
- Em geral $(T_1, T_2, \dots, T_n)$ é o tipo de tuplas com n valores de tipos T_i para i de 1 a n .
- **Tupla vazia** é escrita como `()` e não há tuplas de um único elemento por questão de sintaxe, por exemplo `('a')`.

Definição de funções em Haskell

- Os nomes de funções e argumentos devem começar por letras **minúsculas** e podem incluir letras, dígitos, sublinhados e apóstrofos, exemplo:

`fun1      x_2      y'      fooBar`

- As seguintes palavras reservadas não podem ser usadas como identificadores:

`case class data default deriving do else if import in infix
infixl infixr instance let module newtype of then type where`

- Uma **função** é um mapeamento de valores de um tipo (domínio) em valores de outro tipo (contra-domínio). Em geral:

$t_1 \rightarrow t_2$

funções mapeia valores do tipo t_1 em valores do tipo t_2 .

Definição de funções em Haskell

- A definição de uma função deve obedecer a sintaxe seguinte:

```
nome_função :: tipo_arg1 -> ... -> tipo_argN -> tipo_retorno  
nome_função arg1 ... argN = <expressão>
```

Os `tipo_arg1, ..., tipo_argN` são os tipos dos parâmetros de entrada da função

- Declaração da função `soma` que recebe um `Int`, depois outro `Int`, retorna `Int`

```
-- calcula a soma entre dois numeros
```

```
soma :: Int -> Int -> Int
```

```
soma x y = x + y
```

Definição de funções em Haskell

- Para executar a função o ***runtime*** do Haskell procura por um identificador chamado `main` para iniciar o programa.

```
main :: IO () -- main é uma ação de entrada/saída (IO)
```

```
main = do print(soma 10 20) -- do serve para sequenciar ações IO.
```

Funções polimórficas

- Quando não definimos o tipo de função ela pode operar com valores de qualquer tipo, desde que as operações realizadas pela função seja compatível com o tipo informado pelos valores dos argumentos.

- Para formalizar o tipo variável podemos declarar uma **função** como **polimorfa**.

```
soma :: Num a => a -> a -> a
```

```
soma x y = x + y
```

- “`Num a =>`” é uma restrição a variável `a`, que indica que a função `soma` opera apenas sobre tipos `a` que sejam numéricos.

```
main :: IO ()
```

```
main = do print(soma 10.1 20)
```

Fluxo de condicional if-then-else

- Para calcular a resposta de uma função podemos usar a uma estrutura condicional **if**, a *condição* é uma **expressão booleana** (chamada **predicado**) e exp_1 (chamada **consequência**) e exp_2 (chamada **alternativa**) são expressões de um **mesmo tipo**.
- A resposta da função é o valor de exp_1 se a *condição* é verdadeira, ou o valor de exp_2 se a *condição* é falsa. Importante: a alternativa **else** é obrigatória.

$f p_1 \dots p_n = \text{if } \textit{condição} \text{ then } exp_1 \text{ else } exp_2$

- Uma função para encontrar o menor entre dois números seria:
menor :: Int -> Int -> Int
menor x y = if x <= y then x else y

Fluxo de condicional com guarda (switch..case)

- Uma função pode ser definidas através de **equações com guardas**. Uma **guarda** é formada por uma **sequência de cláusulas** escritas logo após a lista de argumentos.
- Cada **cláusula** tem barra vertical (|=pipe) e uma *condição* e uma expressão (*expr*), separados por (=). **otherwise** é uma *condição* que captura todas as outras situações que ainda não foram consideradas. As **guardas** devem ficar todas com o mesmo **alinhamento à direita**.

$$\begin{array}{l} f p_1 \dots p_n \\ \quad | \text{ condição}_1 = \text{exp}_1 \\ \quad \vdots \\ \quad | \text{ condição}_n = \text{exp}_n \\ \quad | \text{ otherwise } = \text{exp}_{\text{caso contrário}} \end{array}$$

Funções com guarda

- Considere a função para calcular o menor entre 3 números com if/else.

```
menor_tres :: Int -> Int -> Int -> Int
```

```
menor_tres x y z =  
    if x <= y && x <= z then x  
    else if y <= z then y  
    else z
```

- veja a função para calcular o menor entre 3 números com **guarda**.

```
menor_tres :: Int -> Int -> Int -> Int
```

```
menor_tres x y z  
    | x <= y && x <= z = x  
    | y <= z = y  
    | otherwise = z
```

Funções com where

- Em Haskell é possível definir **valores** e **funções auxiliares** em uma definição principal de uma função.
- Isto pode ser feito escrevendo-se uma cláusula **where** ao final de uma cláusula com **guarda** ou para acrescentar parâmetros a uma função.
- A cláusula **where** faz definições que são locais à definição principal de uma função, ou seja, o escopo dos nomes definidos em uma cláusula **where** restringe-se a somente a função principal.

Funções com where

- Um exemplo mostra uma função para análise do índice de massa corporal.

```
calc_imc :: Double -> Double -> String
calc_imc peso altura
    | imc <= 18.5 = "Abaixo do peso!"
    | imc <= 25.0 = "Peso ideal!"
    | imc <= 30   = "Acima do peso!"
    | otherwise  = "Muito acima do peso!"
  where imc = peso / altura ^ 2
```

```
main = do print(calc_imc 70.5 1.72)
```


Exercícios condicionais

- 1) Escreva uma função que recebe um número e verifica se ele é par. A função retorna **True** caso o número seja par ou **False** caso contrário.
- 2) Usando a função par escreva a função impar que verifica se um número é impar utilizando a função par para calcular a resposta, o resultado da função par é negado com o operado **not** do Haskell,
- 3) Dado um n , escreva uma função que devolva a diferença absoluta entre n e 21, exceto devolva o dobro da diferença absoluta se n for maior que 21.
- 4) Faça uma função que recebe três valores inteiros e retorna o maior deles.

Exercícios condicionais

5) (beecrowd | 1047) Crie uma função que receba a hora inicial, minuto inicial, hora final e minuto final de um jogo. A seguir calcule a função calcula duração do jogo.

Obs: O jogo tem duração mínima de um (1) minuto e duração máxima de 24 horas.

Exemplo: se for informado hora inicial=7, minuto inicial=8, hora final=9 e minuto final=10 o seu programa teria como saída O JOGO DUROU 2 HORA(S) E 2 MINUTO(S)

Exercícios condicionais

- 6) Escreva uma função que receba três valores para os lados de um triângulo. Na função verifique se os lados fornecidos realmente formam um triângulo, se formarem devolva tipo de triângulo temos: “isósceles”, “escaleno” ou “equilátero”. Abaixo a declaração da função.

```
triangulo :: Int -> Int -> Int -> String
```

- 7) Escreva uma função que receba 3 valores quaisquer e verifique se os valores podem ser considerados uma tripla de Pitágoras, ou seja, a soma dos quadrados de dois números é igual ao quadrado terceiro. Caso tenhamos uma tripla de Pitágoras a função devolve “eh uma tripla de Pitagoras” e caso não seja o a função devolve “nao eh tripla de Pitagoras”. Exemplos:

3 5 4 é uma tripla de Pitágoras

5 3 4 é uma tripla de Pitágoras

2 4 3 Não é tripla de Pitágoras

Exercícios condicionais

8) No correio local há somente selos de 3 e de 5 centavos. A taxa mínima para correspondência é de 8 centavos. Faça uma função que determina o menor número de selos de 3 e de 5 centavos que completam o valor de uma taxa dada.

ESSA É IMPORTANTE:

`selos :: Int -> (Int, Int)`

9) Dado um número não negativo, suponha **num**. Escreva uma função que devolva **True** se **num** estiver a **2 unidades** de um múltiplo de **10**. Por exemplo para **17** é retornado **False** e para 12 e 19 **True**.

10) Dados três valores inteiros, **a b c**, Escreva uma função que retorne a soma deles. No entanto, se um dos valores for igual a outro, ele não será contabilizado na soma. Por exemplo **a=3, b=2, c=3** a função retorna **2** e para **a=3, b=3, c=3** a função retorna **0**. E por fim para **a=1, b=2, c=3** a função retorna **6**.

Exercícios

11) Considere as expressões lambda em Haskell abaixo, escreva funções no paradigma imperativo (usando a linguagem C) que faz a mesma coisa que as expressões, ou seja para uma mesma entrada a expressão e função tem a mesma saída.

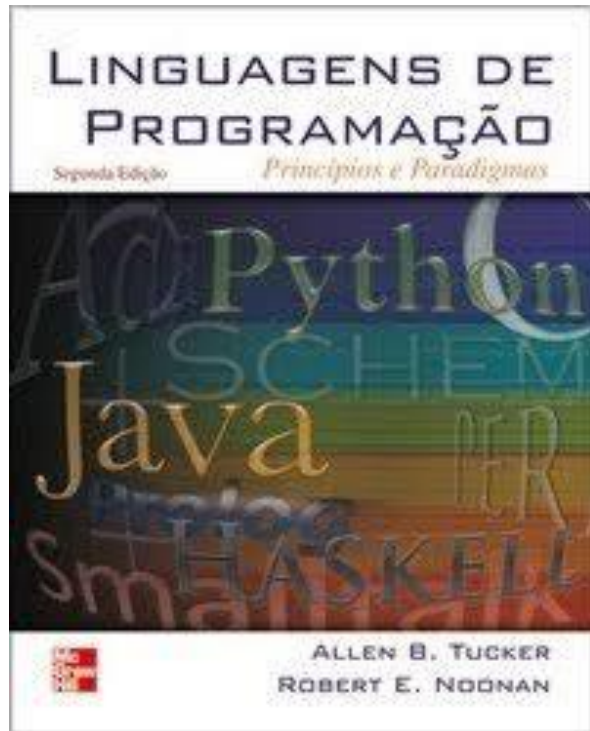
a) `(\x y z -> if x>=y && x>=z then x else if y>=z then y else z)`

b) `(\x min max -> if x<min then min else if x>max then max else x)`

c) `funcao :: Int -> Int -> Int -> Bool`

```
funcao a b c = if a*a + b*b == c*c then True  else if a*a +  
c*c == b*b then True  else if b*b + c*c == a*a then True  else  
False
```

Leituras recomendadas



TUCKER, A. B.; NOONAN, R. E.
**Linguagens de programação:
Princípios e Paradigmas.**
Capítulo 14 sessões 14.1 e 14.3

Dúvidas ??

Obrigado !